# Immutable Collections & Modern APIs

List.of(), Map.of(); new String methods

Rahul Guha || SME || Java FS Training || HCL

2. Modern Java (8–21)

# Immutable Collections in Java

An immutable collection is a collection whose contents cannot be modified after creation. Java introduced convenient factory methods such as List.of(), Set.of(), and Map.of() in Java 9.
Why use immutable collections?
1. Prevent accidental modification of data. 2. Make configuration and constant data safer.
3. Reduce defensive copying in suitable situations. 4.Improve code readability. 5. Help create predictable APIs.
**Mutable vs. immutable collections**

| Feature | Mutable collection | Immutable collection |
|---|---|---|
| Add elements | Allowed | Not allowed |
| Remove elements | Allowed | Not allowed |
| Replace elements | Allowed | Not allowed |
| Example | new ArrayList<>() | List.of() |
| Typical use | Data that changes | Fixed data |
| Modification result | Collection changes | Throws UnsupportedOperationException |

# Before Java 9 : Creating Unmodifiable Collections

Before Java 9, Collections.unmodifiable() methods are used to create unmodifiable collections. These methods just behave like wrapper methods which return unmodifiable view or read-only view of the original collection. i.e you can't perform modifying operations like add, remove, replace, clear etc through the references returned by these wrapper methods. But, you can modify original collection if you have other references to it and those modifications will be reflected in the view returned by these methods.

For example, in the below program, unModifiableSportList is created from sportList through Collections.unmodifiableList(). unModifiableSportList just acts as a read-only view of the original sportList. You can't add elements to unModifiableSportList. If you try to add, it will give UnsupportedOperationException. But, you can add elements to original sportList and those elements will be reflected in unModifiableSportList.

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
public class Java9ImmutableCollections
{  public static void main(String[] args)
   {
      List<String> sportList = new ArrayList<String>();
      sportList.add("Hockey");
      sportList.add("Cricket");
      sportList.add("Tennis");
      List<String> unModifiableSportList =
Collections.unmodifiableList(sportList);
       System.out.println(sportList);    //Output : [Hockey, Cricket,
Tennis]
      System.out.println(unModifiableSportList);    //Output :
[Hockey, Cricket, Tennis]
      unModifiableSportList.add("Wrestling");     //It gives run-time
error
      sportList.add("Kabaddi");      //It gives no error and will be
reflected in unModifiableSportList
      System.out.println(sportList);    //Output : [Hockey, Cricket,
Tennis, Kabaddi]
      System.out.println(unModifiableSportList);    //Output :
[Hockey, Cricket, Tennis, Kabaddi]

   }}
```

# Java 9 Immutable Collections

1) Immutable List:

Immutable List is created by calling List.of() method. This method has other overloaded forms to facilitate the creation of immutable list with desired number of elements. They are as follows. Learnto Code.

**//Returns immutable list with zero element.**
**of()**

**//Returns immutable list with one element.**
**of(E e1)**

**//Returns immutable list with two elements.**
**of(E e1, E e2)**

**//Returns immutable list with three elements.**
**of(E e1, E e2, E e3)**

**//Returns immutable list with four elements.**
**of(E e1, E e2, E e3, E e4)**

**//Returns immutable list with five elements.**
**of(E e1, E e2, E e3, E e4, E e5)**

//Returns immutable list with six elements.
of(E e1, E e2, E e3, E e4, E e5, E e6)
//Returns immutable list with seven elements.
of(E e1, E e2, E e3, E e4, E e5, E e6, E e7)
//Returns immutable list with eight elements.
of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8)
//Returns immutable list with nine elements.
of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8, E e9)
//Returns immutable list with ten elements.
of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8, E e9, E e10)
//Returns immutable list with arbitrary number of elements.
of(E… elements)

# Java 9 Immutable Collections

2) Immutable Set

Immutable set is created by invoking Set.of() method. This method also has several overloaded versions to create immutable set with desired number of elements.**elements.**

**of(E e1, E e2, E e3, E e4, E e5)**

```
//Returns immutable set with zero element.
of()

//Returns immutable set with one element.
of(E e1)

//Returns immutable set with two elements.
of(E e1, E e2)

//Returns immutable set with three elements.
of(E e1, E e2, E e3)

//Returns immutable set with four elements.
of(E e1, E e2, E e3, E e4)

//Returns immutable set with five elements.
of(E e1, E e2, E e3, E e4, E e5)
```

```
//Returns immutable set with six elements.
of(E e1, E e2, E e3, E e4, E e5, E e6)
//Returns immutable set with seven elements.
of(E e1, E e2, E e3, E e4, E e5, E e6, E e7)
//Returns immutable set with eight elements.
of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8)
//Returns immutable set with nine elements.
of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8, E e9)
//Returns immutable set with ten elements.
of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8, E e9, E e10)
//Returns immutable set with arbitrary number of elements.
of(E… elements)
```

# Java 9 Immutable Collections

| Before Java 9 | After Java 9 |
|---|---|
| **1) Immutable List :**<br><br>`List<String> sportList = new ArrayList<String>();`<br><br>`sportList.add("Hockey");`<br>`sportList.add("Cricket");`<br>`sportList.add("Tennis");`<br><br>`List<String> unModifiableSportList = Collections.unmodifiableList(sportList);` | `List<String> immutableSportList = List.of("Hockey", "Cricket", "Tennis");` |
| **2) Immutable Set :**<br><br>`Set<String> sportSet = new HashSet<>();`<br><br>`sportSet.add("Hockey");`<br>`sportSet.add("Cricket");`<br>`sportSet.add("Tennis");`<br><br>`Set<String> unModifiableSportSet = Collections.unmodifiableSet(sportSet);` | `Set<String> immutableSportSet = Set.of("Hockey", "Cricket", "Tennis");` |
| **3) Immutable Map :**<br><br>`Map<Integer, String> sportMap = new HashMap<>();`<br><br>`sportMap.put(1, "Hockey");`<br>`sportMap.put(2, "Cricket");`<br>`sportMap.put(3, "Tennis");`<br><br>`Map<Integer, String> unModifiableSportMap = Collections.unmodifiableMap(sportMap);` | `Map<Integer, String> immutableSportMap = Map.of(1, "Hockey", 2, "Cricket", 3, "Tennis");` |

**Immutable Vs Unmodifiable :**

**Java 9 Immutable collections and unmodifiable collections returned by the Collections.unmodifiableXXX() wrapper methods are not the same. Unmodifiable collections are just the read-only views of the original collection. You can perform modifying operations on the original collection and those modifications will be reflected in the collections returned by these methods. But, immutable collections returned by Java 9 static factory methods are 100% immutable. You can't modify them once they are created.**

# Characteristics Of Java 9 Immutable Collections :

1. **Modifying operations on immutable collections are not allowed. If you try to modify them, UnsupportedOperationException will be thrown.**
2. **Null elements are not allowed. They will give NullPointerException at run time.**
3. **Java 9 immutable collections are thread safe. You can use them in a multi threaded environment without synchronization.**
4. **Immutable collections returned by Java 9 static factory methods are space efficient. They consume less memory than the mutable collections.**

**Where to use them?**

You can consider using Java 9 immutable collections if you have lots of known values and those values never change throughout the execution and those values are retrieved frequently. In such scenarios, immutable collections give better performance than mutable collections.

# AI Integration

Generating stream pipelines with Copilot; validating AI-generated functional code for correctness and readability

Rahul Guha || SME || Java FS Training || HCL

2. Modern Java (8–21)

# AI Integration

**Step-by-Step Guide:** Generating Java Stream Pipelines with GitHub CopilotThis practical guide will help you use GitHub Copilot in VS Code to generate Java Stream API code from natural-language prompts, execute the generated program, and verify that the output is correct.We will build one complete Java program using filter(), map(), sorted(), and toList().

**Step 1: Set up your development environment**

You need the following tools.

- Visual Studio Code
- GitHub Copilot
- JDK 17 or later

**Install the required VS Code extensions**

1. Open VS Code.
2. Click the Extensions icon in the left sidebar.
3. Search for Extension Pack for Java.
4. Install the Java extension pack.

**5. Search for GitHub Copilot and install the official extension if it is not already installed.**
**6. Sign in with your GitHub account and complete any required authorization.**
**Extension availability and Copilot features may depend on your account plan and organization settings.**

# AI Integration

- **Step 2: Verify Java installation**
  Open the VS Code terminal using Terminal → New Terminal.
- java -version
- javac -version
  Both commands should display installed Java version information.For this exercise, use Java 17 or later so that Stream.toList() is available.

**Step 3: Create a Java project**

1. Create a folder named CopilotStreamDemo.
2. Open the folder in VS Code.
3. Create a file named StreamPipelineDemo.java.
   Your project structure should look like this:

**CopilotStreamDemo/**
**└── StreamPipelineDemo.java**

**Step 4: Understand the problem before prompting AI**

We want to process a list of employee salaries.

**Problem statement:**

Given salaries [30000, 60000, 45000, 80000, 55000]:
- Select salaries greater than ₹50,000.
- Increase each selected salary by 10%.
- Sort the updated salaries in ascending order.
**- Display the final list.**
**[60500, 66000, 88000]**

Defining the expected output before asking Copilot is important because it gives you a reference for checking the generated solution.

# AI Integration

- **Step 5: Generate the Stream pipeline using Copilot**
  **Open** StreamPipelineDemo.java.
  There are two convenient ways to generate code.

  ## Method A: Use Copilot Chat

  1. Open Copilot Chat from the VS Code sidebar or use the available Copilot Chat shortcut.
  2. Select the appropriate chat mode for asking questions or generating code.
  3. Copy and submit this prompt.
  4. Read the generated code before accepting it.
  5. Copy or insert the code into : **StreamPipelineDemo.java.**
  6. Save the file.

**Prompt 1 — Generate Java Stream code**
**Write a complete Java 17 program named StreamPipelineDemo.**

**Requirements:**

1. Create a list of employee salaries: 30000, 60000, 45000, 80000, 55000.
2. Use the Stream API to filter salaries greater than 50000.
3. Increase each selected salary by 10%.
4. Sort the updated salaries in ascending order.
5. Collect the results into a list and print them.
6. Use meaningful variable names.
7. Explain the purpose of each Stream operation.
8. Show the expected output.

# AI Integration

- **Method B: Use inline code generation**
  **You can also type a descriptive comment directly in the Java file:**

- // Generate a Stream pipeline that filters salaries above
- // 50000, increases them by 10%, sorts them, and prints
- // the resulting list.

Wait for Copilot's suggestion, review it, and accept it only if it matches the requirement. If no suggestion appears, use Copilot Chat instead.

**Prompt 1 — Generate Java Stream code**
**Write a complete Java 17 program named StreamPipelineDemo.**
**Requirements:**
1. Create a list of employee salaries: 30000, 60000, 45000, 80000, 55000.
2. Use the Stream API to filter salaries greater than 50000.
3. Increase each selected salary by 10%.
4. Sort the updated salaries in ascending order.
5. Collect the results into a list and print them.
6. Use meaningful variable names.
7. Explain the purpose of each Stream operation.
8. Show the expected output.

# AI Integration

- **Step 6: Review the generated code**
  **A suitable implementation is:**
  **import java.util.List;**

```
public class StreamPipelineDemo {
    public static void main(String[] args) {
        List<Integer> salaries =
                List.of(30000, 60000, 45000, 80000, 55000);
        List<Integer> updatedSalaries = salaries.stream()
                .filter(salary -> salary > 50000)
                .map(salary -> (int) (salary * 1.10))
                .sorted()
                .toList();
        System.out.println(updatedSalaries);
    }
}
```

- **Expected output:**
  **[60500, 66000, 88000]**
  **Check each operation:**

| Code | Purpose |
|---|---|
| salaries.stream() | Creates a stream from the salary list |
| filter(...) | Selects salaries greater than 50,000 |
| map(...) | Increases each selected salary by 10% |
| sorted() | Sorts the transformed values |
| toList() | Collects results into an unmodifiable list |

The original salary list is not modified.

# AI Integration

## Step 7: Execute the generated program

Open the terminal in the directory containing the Java file.

**Compile the program:**

**javac StreamPipelineDemo.java**

**Run the program:**

java StreamPipelineDemo

**Expected terminal output:**

[60500, 66000, 88000]

If compilation fails, inspect the error message and ask Copilot to explain the error. Do not blindly accept a proposed fix; compile and run the corrected version again.

## Step 8: Ask Copilot to explain the pipeline

After generating the program, submit this follow-up prompt.

**Explain this Java Stream pipeline step by step. Show the intermediate result after:**

1. filter()
2. map()
3. sorted

**()Explain why the operations are performed in this order.**

Do not change the original code.

## The intermediate results should be:

1. filter()---Filtering ---Remove salaries of ₹50,000 or less.[60000, 80000, 55000]
2. map()---Transformation----Apply the 10% increase.[66000, 88000, 60500]
3. sorted()---Ordering---Arrange values in ascending order.[60500, 66000, 88000]

# AI Integration

## Step 9: Validate the AI-generated code

Now use Copilot as a code-review assistant.

**Submit this prompt:**

Review my Java Stream pipeline for:
1. Logical correctness.
2. Correct order of operations.
3. Java version compatibility.
4. Readability and meaningful naming.
5. Edge cases, including an empty list.
6. Possible integer overflow or precision problems. Identify specific issues and explain why they matter. Do not rewrite correct code unnecessarily.

……..

Independently verify Copilot's response:
- Does the filter match the stated requirement?
- Is the increment calculated correctly?

**- Does the output match your manually calculated result?**
**- Does the code compile with your installed JDK?**
**- Are edge cases handled appropriately?**
**For real payroll calculations, use BigDecimal and explicit rounding rules rather than relying on integer multiplication.**

## Step 10: Test the code with different inputs

Modify the salary list and execute the program again.

**Interactive pipeline tester**

Experiment with different integer inputs. The calculation below illustrates the pipeline; verify your Java program separately.

**Input salaries (comma-separated)**
**30000, 60000, 45000, 80000, 55000**

# AI Integration

## Try these test cases:

| Test | Input | Expected Behavior |
|---|---|---|
| Normal | 30000, 60000, 80000 | Keep and increase two salaries |
| Empty | No values | Return an empty list |
| No matches | 10000, 20000 | Return an empty list |
| Boundary | 50000, 50001 | Exclude 50000; include 50001 |
| Duplicates | 60000, 60000 | Retain both results unless distinct() is added |

**Step 11: Improve readability with Copilot**

**Ask Copilot:**

Refactor this Java Stream pipeline for readability without changing its behavior. Use meaningful variable names, avoid unnecessary operations, and explain any trade-offs. Preserve the same output and include a test that verifies equivalence.

Compare the original and revised versions. Accept a refactoring only after verifying that it preserves the required behavior.

## 1. SETUP & PROMPT

Prepare the environment and use Copilot to generate Stream pipeline code

### 1 Install Required Tools

- **Visual Studio Code** — Code editor
- **GitHub Copilot** — AI code assistant
- **Java JDK 17+** — Compiler & runtime

### 2 Install Extensions in VS Code




### 3 Create a Java File

```
CopilotStreamDemo/
    StreamPipelineDemo.java
```

### 4 Define the Problem (Natural Language)

Given salaries [30000, 60000, 45000, 80000, 55000]:

- Select salaries greater than 50,000
- Increase each selected salary by 10%
- Sort the updated salaries in ascending order
- Display the final list

### 5 Use Copilot to Generate Code

**Option A: Copilot Chat**

Write a complete Java 17 program named StreamPipelineDemo. Create a list of salaries 30000, 60000, 45000, 80000, 55000. Use Stream API to filter salaries > 50000, increase each by 10%, sort and print the result. Explain each step.

**Option B: Inline Suggestion**

```
// Generate a Stream pipeline that filters
// salaries above 50000, increases by 10%,
// sorts them, and prints the resulting list.
```

## 2. GENERATED CODE & EXECUTION

Review, understand and run the Copilot-generated Stream pipeline

### 1 Copilot-Generated Java Code (Stream Pipeline)

StreamPipelineDemo.java

```
1  import java.util.List;
2
3  public class StreamPipelineDemo {
4      public static void main(String[] args) {
5          // List of employee salaries
6          List<Integer> salaries =
7              List.of(30000, 60000, 45000, 80000, 55000);
8
9          // Stream pipeline
10         List<Integer> updatedSalaries = salaries.stream()
11             .filter(salary -> salary > 50000)   // 1. filter
12             .map(salary -> (int) (salary * 1.10)) // 2. map
13             .sorted()                           // 3. sorted
14             .toList();                          // 4. collect
15
16         System.out.println(updatedSalaries);
17     }
18 }
```

### 2 Explanation of Stream Operations

| Operation | Purpose |
|---|---|
| `stream()` | Creates a stream from the salary list |
| `filter(salary > 50000)` | Selects salaries greater than 50,000 |
| `map(salary * 1.10)` | Increases each selected salary by 10% |
| `sorted()` | Sorts the updated salaries in ascending order |
| `toList()` | Collects the results into an unmodifiable list (Java 16+) |

### 3 Compile and Run the Program

Terminal

```
javac StreamPipelineDemo.java
java StreamPipelineDemo
```

### 4 Output

Output

```
[60500, 66000, 88000]
```

## 3. VALIDATION, TESTING & FLOW

Verify correctness, improve readability and follow the end-to-end process

### 1 Intermediate Results (Conceptual View)

| Step | Operation | Intermediate Result |
|---|---|---|
| 1 | filter() | [60000, 80000, 55000] |
| 2 | map() | [66000, 88000, 60500] |
| 3 | sorted() | [60500, 66000, 88000] |

### 2 Validate the Generated Code

- ✅ Logical correctness (matches requirement)
- ✅ Correct order of Stream operations
- ✅ Compiles with Java 17 or later
- ✅ Readable and meaningful variable names
- ✅ Handles edge cases (empty list, no matches)
- ✅ Check for possible overflow/precision issues
- ✅ Run the code and verify the output

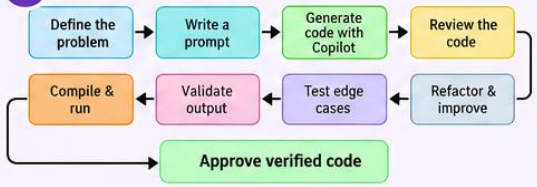
### 3 Test with Different Inputs

| Test Case | Input Salaries | Expected Output |
|---|---|---|
| Normal | [30000, 60000, 80000] | [66000, 88000] |
| Empty list | [] | [] |
| No matches | [10000, 20000] | [] |
| Boundary | [50000, 50001] | [55001] |
| Duplicates | [60000, 60000] | [66000, 66000] |

### 4 Improve Readability (Refactor with Copilot)

Refactor this Java Stream pipeline for readability without changing its behavior. Use meaningful variable names and explain any trade-offs.

### 5 End-to-End Process Flow




## Your practical assignment

Create a Java program that uses Copilot to help generate a Stream pipeline for student marks. Filter passing marks (40 or above), convert them into grades, sort the results, and test boundary cases such as 39, 40, 89, 90, and 100.

**Final takeaway:** Copilot generates suggestions; the developer defines the requirements, checks the logic, runs the tests, and decides whether the code is correct and readable.